Java Swing:
10 ejercicios resueltos

Guía práctica complementaria de la Semana 6 · Diseñando interfaces gráficas con Swing en Java

Cada ejercicio presenta un **enunciado**, los **requisitos**, la **vista esperada del formulario** (captura real de la aplicación en ejecución), la **solución completa** en Java y una explicación de las decisiones de diseño. Las temáticas son distintas en todos los casos. El **ejercicio 7** integra **hilos con un monitor** implementado con un `JTextArea`.

N°	Temática	Ejercicio	Componentes y conceptos clave
1	Salud y nutrición	**Calculadora de IMC**	JFrame, JPanel, JLabel, JTextField, JButton, GridLayout, BorderLayout, ActionListener, NumberFormatException
2	Ciencias y física	**Conversor de temperaturas**	JComboBox, getSelectedIndex(), switch con flechas, validación de rango, requestFocusInWindow()
3	Productividad personal	**Lista de tareas pendientes**	JList, DefaultListModel, JScrollPane, TitledBorder, JOptionPane (mensaje y confirmación)
4	Educación	**Registro de notas de estudiantes**	JTable, DefaultTableModel, isCellEditable, JSpinner, SpinnerNumberModel, JComboBox
5	Seguridad informática	**Login con intentos limitados**	JPasswordField, GridBagLayout, JCheckBox, JOptionPane, setEnabled(), char[]
6	Gastronomía y ventas	**Pedido de pizzería con boleta**	JRadioButton, ButtonGroup, JCheckBox, JSpinner, JTextArea, String.format, TitledBorder
7	Concurrencia y sistemas	**Monitor de hilos con JTextArea**	Thread, Runnable (lambda), synchronized, Object como cerrojo, SwingUtilities.invokeLater, interrupt(), JTextArea, JScrollPane
8	Videojuegos y lógica	**Juego: adivina el número**	Random, JSpinner, JProgressBar, estado del juego, UIManager.getColor(), setEnabled()
9	Ofimática y archivos	**Mini editor de texto**	JMenuBar, JMenu, JMenuItem, JCheckBoxMenuItem, KeyStroke, JFileChooser, Files, DocumentListener
10	Gráficos y animación	**Semáforo animado**	paintComponent, Graphics2D, antialiasing, enum con atributos, switch expression, javax.swing.Timer, repaint()

Cómo ejecutar cada ejercicio

1. En **IntelliJ IDEA** crea una clase con el nombre exacto indicado en el título del código (por ejemplo `Ej01Imc`) y pega la solución completa.
2. Ejecuta con el ícono ▶ junto al método `main`. Cada archivo es independiente y no requiere librerías externas.
3. Por terminal: `javac Ej01Imc.java` y luego `java Ej01Imc`. Se requiere **Java 17 o superior** (usa `switch` con flechas y `Files.readString`).

Todo el código fue compilado y ejecutado para generar las capturas; los valores que aparecen en ellas (por ejemplo, las lecturas del monitor) son resultados reales de la ejecución.

EJERCICIO 01

Calculadora de IMC

Temática: Salud y nutrición

Enunciado

Una nutricionista necesita una ventana simple para calcular el Índice de Masa Corporal (IMC) de sus pacientes. El programa recibe el peso y la altura, calcula el IMC y muestra el resultado junto a su categoría, con un color distinto para cada una.

Requisitos

- Campos de texto para **peso (kg)** y **altura (m)**; deben aceptar coma o punto decimal.
- Fórmula: `IMC = peso / altura²`, mostrado con un decimal.
- Categorías: menor a 18,5 bajo peso; hasta 24,9 normal; hasta 29,9 sobrepeso; desde 30 obesidad.
- Validar que los valores sean numéricos y mayores que 0, sin cerrar el programa si hay un error.

Conceptos que se aplican: JFrame, JPanel, JLabel, JTextField, JButton, GridLayout, BorderLayout, ActionListener, NumberFormatException

Vista esperada del formulario

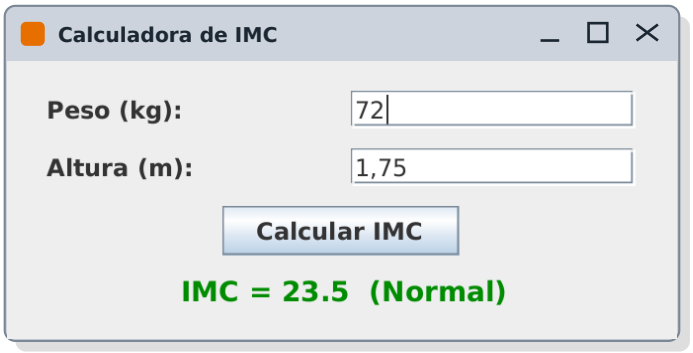


Figura 1. Formulario del ejercicio 1 en ejecución.

Solución · Ej01Imc.java

```
import javax.swing.*;
import java.awt.*;

public class Ej01Imc extends JFrame {

    private final JTextField txtPeso = new JTextField(8);
    private final JTextField txtAltura = new JTextField(8);
    private final JLabel lblResultado =
            new JLabel("Ingrese sus datos y presione Calcular", SwingConstants.CENTER);

    public Ej01Imc() {
        super("Calculadora de IMC");
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        JPanel formulario = new JPanel(new GridLayout(2, 2, 10, 10));
        formulario.setBorder(BorderFactory.createEmptyBorder(15, 20, 5, 20));
        formulario.add(new JLabel("Peso (kg):"));
        formulario.add(txtPeso);
        formulario.add(new JLabel("Altura (m):"));
        formulario.add(txtAltura);

        JButton btnCalcular = new JButton("Calcular IMC");
        btnCalcular.addActionListener(e -> calcular());
        JPanel panelBoton = new JPanel();
        panelBoton.add(btnCalcular);

        lblResultado.setFont(lblResultado.getFont().deriveFont(Font.BOLD, 14f));
        lblResultado.setBorder(BorderFactory.createEmptyBorder(5, 20, 15, 20));
```

```
        add(formulario, BorderLayout.NORTH);
        add(panelBoton, BorderLayout.CENTER);
        add(lblResultado, BorderLayout.SOUTH);

        pack();
        setLocationRelativeTo(null);
    }

    private void calcular() {
        try {
            double peso = Double.parseDouble(txtPeso.getText().trim().replace(',', '.'));
            double altura = Double.parseDouble(txtAltura.getText().trim().replace(',', '.'));
            if (peso <= 0 || altura <= 0) {
                throw new NumberFormatException("Valores no positivos");
            }
            double imc = peso / (altura * altura);

            String categoria;
            Color color;
            if (imc < 18.5) {
                categoria = "Bajo peso";
                color = new Color(0, 102, 204);
            } else if (imc < 25) {
                categoria = "Normal";
                color = new Color(0, 140, 0);
            } else if (imc < 30) {
                categoria = "Sobrepeso";
                color = new Color(210, 120, 0);
            } else {
                categoria = "Obesidad";
                color = new Color(190, 0, 0);
            }
            lblResultado.setForeground(color);
            lblResultado.setText(String.format("IMC = %.1f  (%s)", imc, categoria));
        } catch (NumberFormatException ex) {
            lblResultado.setForeground(Color.RED);
            lblResultado.setText("Ingrese números válidos mayores que 0");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Ej01Imc().setVisible(true));
    }
}
```

¿Cómo funciona?

- Los componentes son **atributos de la clase** para que `calcular()` pueda leerlos; se crean una sola vez.
- Se combinan tres layouts: `GridLayout` para el formulario, el `FlowLayout` por defecto de `JPanel` para centrar el botón y `BorderLayout` para ubicar las tres zonas del frame.
- `replace(',', '.')` permite escribir 1,75 o 1.75. Si el texto no es un número, `parseDouble` lanza `NumberFormatException`, que se captura para mostrar un mensaje en rojo.
- El color del resultado cambia con `setForeground` según la categoría, un feedback visual inmediato.

EJERCICIO 02

Conversor de temperaturas

Temática: Ciencias y física

Enunciado

Un laboratorio escolar requiere un conversor que transforme temperaturas entre Celsius, Fahrenheit y Kelvin. El usuario elige el tipo de conversión en una lista desplegable, escribe el valor y obtiene el resultado.

Requisitos

- `JComboBox` con las cuatro conversiones: C a F, F a C, C a K y K a C.
- Botón **Convertir** y botón **Limpiar** (deja el foco listo en el campo de texto).
- Validar número y rechazar valores bajo el cero absoluto (−273,15 °C, −459,67 °F, 0 K).
- Mostrar el resultado con dos decimales y su unidad.

Conceptos que se aplican: JComboBox, getSelectedIndex(), switch con flechas, validación de rango, requestFocusInWindow()

Vista esperada del formulario



Figura 2. Formulario del ejercicio 2 en ejecución.

Solución · Ej02Temperatura.java

```
import javax.swing.*;
import java.awt.*;

public class Ej02Temperatura extends JFrame {

    private final JComboBox<String> cboConversion = new JComboBox<>(new String[]{
            "Celsius a Fahrenheit", "Fahrenheit a Celsius",
            "Celsius a Kelvin", "Kelvin a Celsius"});
    private final JTextField txtValor = new JTextField(10);
    private final JLabel lblResultado = new JLabel(" ", SwingConstants.CENTER);

    public Ej02Temperatura() {
        super("Conversor de temperaturas");
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        JPanel formulario = new JPanel(new GridLayout(2, 2, 10, 10));
        formulario.setBorder(BorderFactory.createEmptyBorder(15, 20, 5, 20));
        formulario.add(new JLabel("Conversión:"));
        formulario.add(cboConversion);
        formulario.add(new JLabel("Valor a convertir:"));
        formulario.add(txtValor);

        JButton btnConvertir = new JButton("Convertir");
        JButton btnLimpiar = new JButton("Limpiar");
        btnConvertir.addActionListener(e -> convertir());
        btnLimpiar.addActionListener(e -> {
            txtValor.setText("");
            lblResultado.setText(" ");
```

```
            txtValor.requestFocusInWindow();
        });
        JPanel botones = new JPanel();
        botones.add(btnConvertir);
        botones.add(btnLimpiar);

        lblResultado.setFont(lblResultado.getFont().deriveFont(Font.BOLD, 20f));
        lblResultado.setBorder(BorderFactory.createEmptyBorder(5, 20, 15, 20));

        add(formulario, BorderLayout.NORTH);
        add(botones, BorderLayout.CENTER);
        add(lblResultado, BorderLayout.SOUTH);

        pack();
        setLocationRelativeTo(null);
    }

    private boolean bajoCeroAbsoluto(int tipo, double v) {
        return ((tipo == 0 || tipo == 2) && v < -273.15)
                || (tipo == 1 && v < -459.67)
                || (tipo == 3 && v < 0);
    }

    private void convertir() {
        try {
            double v = Double.parseDouble(txtValor.getText().trim().replace(',', '.'));
            int tipo = cboConversion.getSelectedIndex();
            if (bajoCeroAbsoluto(tipo, v)) {
                lblResultado.setForeground(Color.RED);
                lblResultado.setText("Valor bajo el cero absoluto");
                return;
            }
            double r;
            String unidad;
            switch (tipo) {
                case 0 -> { r = v * 9 / 5 + 32;  unidad = "°F"; }
                case 1 -> { r = (v - 32) * 5 / 9; unidad = "°C"; }
                case 2 -> { r = v + 273.15;       unidad = "K"; }
                default -> { r = v - 273.15;      unidad = "°C"; }
            }
            lblResultado.setForeground(new Color(0, 90, 160));
            lblResultado.setText(String.format("%.2f %s", r, unidad));
        } catch (NumberFormatException ex) {
            lblResultado.setForeground(Color.RED);
            lblResultado.setText("Ingrese un número válido");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Ej02Temperatura().setVisible(true));
    }
}
```

¿Cómo funciona?

- `getSelectedIndex()` entrega la posición elegida en el combo; un `switch` con flechas (`->`) evita `break` y hace el código más legible.
- Cada rama del `switch` asigna `r` y `unidad`, por lo que Java garantiza que ambas queden inicializadas antes de usarse.
- El método auxiliar `bajoCeroAbsoluto()` separa la regla de negocio de la interfaz, una pequeña muestra de la filosofía MVC.
- `Limpiar` reinicia campos y devuelve el foco al `JTextField` con `requestFocusInWindow()`.

EJERCICIO 03

Lista de tareas pendientes

Temática: Productividad personal

Enunciado

Construye un administrador de tareas: el usuario escribe una tarea, la agrega a una lista visible, marca como completadas las que termina y ve cuántas le quedan pendientes.

Requisitos

- Campo de texto y botón **Agregar** (también debe funcionar al presionar Enter).
- `JList` con selección simple dentro de un `JScrollPane`.
- Botón **Completar tarea** (quita la seleccionada) y **Vaciar lista** (pide confirmación).
- Contador de tareas pendientes siempre actualizado y aviso si se intenta agregar un texto vacío.

Conceptos que se aplican: JList, DefaultListModel, JScrollPane, TitledBorder, JOptionPane (mensaje y confirmación)

Vista esperada del formulario

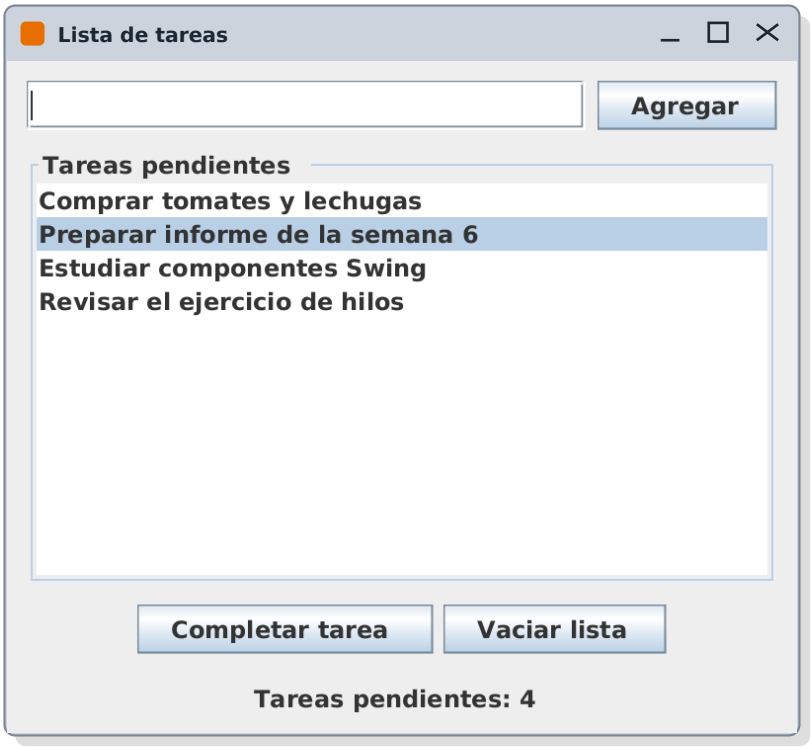


Figura 3. Formulario del ejercicio 3 en ejecución.

Solución · Ej03Tareas.java

```
import javax.swing.*;
import java.awt.*;

public class Ej03Tareas extends JFrame {

    private final DefaultListModel<String> modelo = new DefaultListModel<>();
    private final JList<String> lista = new JList<>(modelo);
    private final JTextField txtTarea = new JTextField();
    private final JLabel lblContador = new JLabel("", SwingConstants.CENTER);

    public Ej03Tareas() {
        super("Lista de tareas");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(400, 340);
        setLocationRelativeTo(null);
```

```
        JButton btnAgregar = new JButton("Agregar");
        JPanel norte = new JPanel(new BorderLayout(6, 0));
        norte.setBorder(BorderFactory.createEmptyBorder(10, 10, 0, 10));
        norte.add(txtTarea, BorderLayout.CENTER);
        norte.add(btnAgregar, BorderLayout.EAST);

        lista.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        JScrollPane scroll = new JScrollPane(lista);
        scroll.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createEmptyBorder(10, 10, 5, 10),
                BorderFactory.createTitledBorder("Tareas pendientes")));

        JButton btnCompletar = new JButton("Completar tarea");
        JButton btnVaciar = new JButton("Vaciar lista");
        JPanel botones = new JPanel();
        botones.add(btnCompletar);
        botones.add(btnVaciar);
        JPanel sur = new JPanel(new GridLayout(2, 1));
        sur.add(botones);
        sur.add(lblContador);

        add(norte, BorderLayout.NORTH);
        add(scroll, BorderLayout.CENTER);
        add(sur, BorderLayout.SOUTH);

        btnAgregar.addActionListener(e -> agregar());
        txtTarea.addActionListener(e -> agregar());          // Enter en el campo de texto
        btnCompletar.addActionListener(e -> completar());
        btnVaciar.addActionListener(e -> vaciar());
        actualizarContador();
    }

    private void agregar() {
        String texto = txtTarea.getText().trim();
        if (texto.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Escriba una tarea antes de agregar.",
                    "Campo vacío", JOptionPane.WARNING_MESSAGE);
            return;
        }
        modelo.addElement(texto);
        txtTarea.setText("");
        txtTarea.requestFocusInWindow();
        actualizarContador();
    }

    private void completar() {
        int indice = lista.getSelectedIndex();
        if (indice < 0) {
            JOptionPane.showMessageDialog(this, "Seleccione una tarea de la lista.");
            return;
        }
        modelo.remove(indice);
        actualizarContador();
    }

    private void vaciar() {
        if (modelo.isEmpty()) return;
        int op = JOptionPane.showConfirmDialog(this, "¿Eliminar todas las tareas?",
                "Confirmar", JOptionPane.YES_NO_OPTION);
        if (op == JOptionPane.YES_OPTION) {
            modelo.clear();
            actualizarContador();
        }
    }

    private void actualizarContador() {
        lblContador.setText("Tareas pendientes: " + modelo.size());
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Ej03Tareas().setVisible(true));
    }
}
```

¿Cómo funciona?

- `JList` solo **muestra**; los datos viven en el `DefaultListModel`. Es la misma separación vista/modelo que `JTable` + `DefaultTableModel` en la semana 6.
- `txtTarea.addActionListener(...)` responde a la tecla Enter, sin necesidad de un `KeyListener`.
- `getSelectedIndex()` devuelve −1 si no hay selección; siempre hay que validarlo antes de `remove()`.
- `JOptionPane.showConfirmDialog` devuelve la opción elegida, lo que permite pedir confirmación antes de una acción destructiva.

EJERCICIO **04**

Registro de notas de estudiantes

Temática: Educación

Enunciado

Un docente quiere registrar el nombre, la carrera y la nota (escala 1,0 a 7,0) de sus estudiantes en una tabla, eliminar registros erróneos y ver el promedio del curso en tiempo real.

Requisitos

- Formulario con `JTextField` (nombre), `JComboBox` (carrera) y `JSpinner` (nota de 1,0 a 7,0).
- `JTable` de solo lectura, alimentada por un `DefaultTableModel`.
- Botones **Agregar** y **Eliminar seleccionado**; el nombre es obligatorio.
- Barra inferior con la cantidad de estudiantes y el promedio (calculado desde el modelo).

Conceptos que se aplican: JTable, DefaultTableModel, isCellEditable, JSpinner, SpinnerNumberModel, JComboBox

Vista esperada del formulario

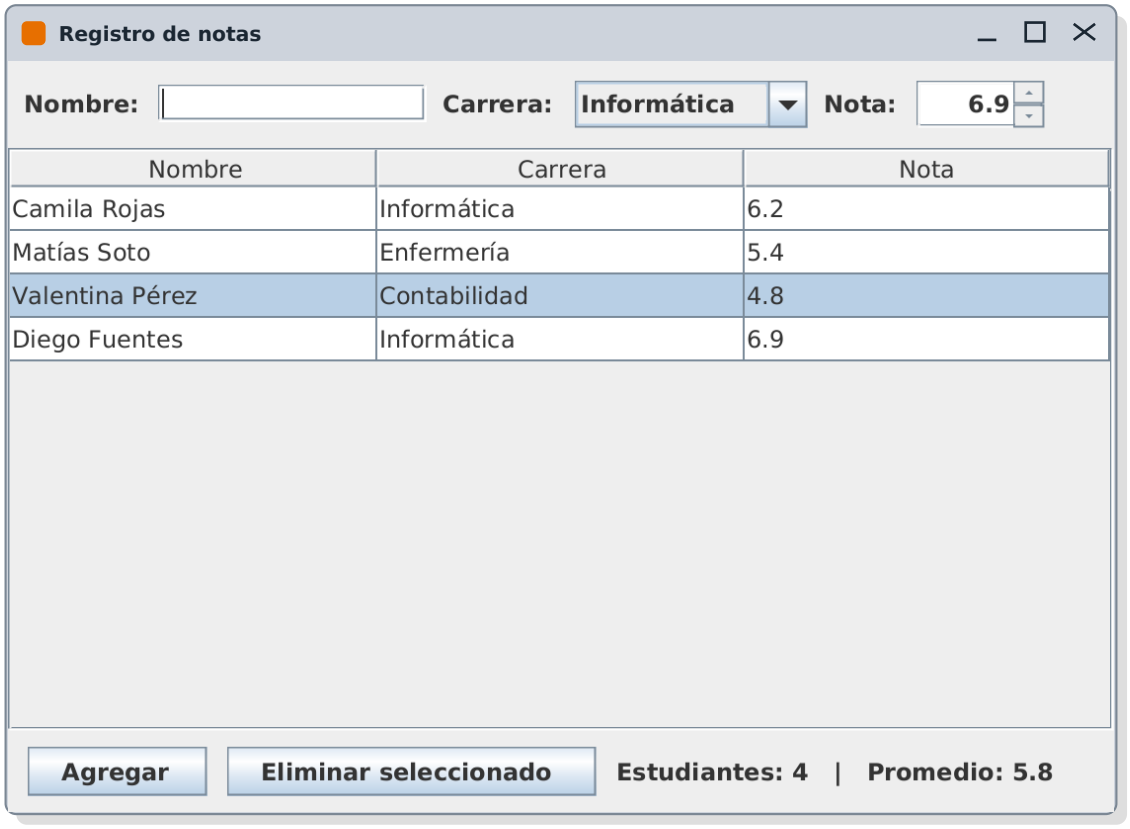


Figura 4. Formulario del ejercicio 4 en ejecución.

Solución · Ej04Estudiantes.java

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class Ej04Estudiantes extends JFrame {

    private final JTextField txtNombre = new JTextField(12);
    private final JComboBox<String> cboCarrera = new JComboBox<>(
            new String[]{"Informática", "Enfermería", "Contabilidad", "Diseño"});
    private final JSpinner spnNota = new JSpinner(new SpinnerNumberModel(4.0, 1.0, 7.0, 0.1));
    private final DefaultTableModel modelo = new DefaultTableModel(
            new String[]{"Nombre", "Carrera", "Nota"}, 0) {
        @Override
        public boolean isCellEditable(int fila, int columna) {
            return false;                       // la tabla es solo de lectura
        }
    };
    private final JTable tabla = new JTable(modelo);
    private final JLabel lblPromedio = new JLabel();

    public Ej04Estudiantes() {
        super("Registro de notas");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(560, 380);
        setLocationRelativeTo(null);

        JPanel formulario = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 10));
        formulario.add(new JLabel("Nombre:"));
        formulario.add(txtNombre);
        formulario.add(new JLabel("Carrera:"));
        formulario.add(cboCarrera);
        formulario.add(new JLabel("Nota:"));
        spnNota.setPreferredSize(new Dimension(65, 24));
        formulario.add(spnNota);

        JButton btnAgregar = new JButton("Agregar");
        JButton btnEliminar = new JButton("Eliminar seleccionado");
        JPanel sur = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 8));
        sur.add(btnAgregar);
        sur.add(btnEliminar);
        sur.add(lblPromedio);

        tabla.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        tabla.setRowHeight(22);

        add(formulario, BorderLayout.NORTH);
        add(new JScrollPane(tabla), BorderLayout.CENTER);
        add(sur, BorderLayout.SOUTH);

        btnAgregar.addActionListener(e -> agregar());
        btnEliminar.addActionListener(e -> eliminar());
        actualizarPromedio();
    }

    private void agregar() {
        String nombre = txtNombre.getText().trim();
        if (nombre.isEmpty()) {
            JOptionPane.showMessageDialog(this, "El nombre es obligatorio.",
                    "Validación", JOptionPane.WARNING_MESSAGE);
            return;
        }
        double nota = Math.round(((Double) spnNota.getValue()) * 10) / 10.0;
        modelo.addRow(new Object[]{nombre, cboCarrera.getSelectedItem(), nota});
        txtNombre.setText("");
        txtNombre.requestFocusInWindow();
        actualizarPromedio();
    }

    private void eliminar() {
        int fila = tabla.getSelectedRow();
        if (fila < 0) {
            JOptionPane.showMessageDialog(this, "Seleccione una fila de la tabla.");
            return;
```

```
        }
        modelo.removeRow(fila);
        actualizarPromedio();
    }

    private void actualizarPromedio() {
        int n = modelo.getRowCount();
        double suma = 0;
        for (int i = 0; i < n; i++) {
            suma += (Double) modelo.getValueAt(i, 2);
        }
        String promedio = (n == 0) ? "-" : String.format("%.1f", suma / n);
        lblPromedio.setText("Estudiantes: " + n + "   |   Promedio: " + promedio);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Ej04Estudiantes().setVisible(true));
    }
}
```

¿Cómo funciona?

- La clase anónima sobre `DefaultTableModel` sobrescribe `isCellEditable` para devolver `false`: la tabla solo se modifica por código.
- `SpinnerNumberModel(4.0, 1.0, 7.0, 0.1)` fija valor inicial, mínimo, máximo y paso, así el usuario no puede ingresar notas inválidas.
- `Math.round(x * 10) / 10.0` elimina errores de punto flotante (4.1000000000000005) antes de guardar la nota.
- El promedio se recalcula recorriendo el modelo con `getValueAt`, por lo que siempre coincide con lo que muestra la tabla.

EJERCICIO
05

Login con intentos limitados

Temática: Seguridad informática

Enunciado

Implementa una ventana de acceso que valide usuario y contraseña. Tras tres intentos fallidos el sistema debe bloquear el formulario. El usuario correcto es `admin` y la clave `1234`.

Requisitos

- Campos `JTextField` y `JPasswordField`, con casilla **Mostrar clave**.
- Mensaje con los intentos restantes tras cada error, en color rojo.
- Al tercer error: deshabilitar campos y botón, y mostrar “Cuenta bloqueada”.
- Acceso correcto: `JOptionPane` de bienvenida (aquí se abriría la ventana principal).

Conceptos que se aplican: JPasswordField, GridBagLayout, JCheckBox, JOptionPane, setEnabled(), char[]

Vista esperada del formulario

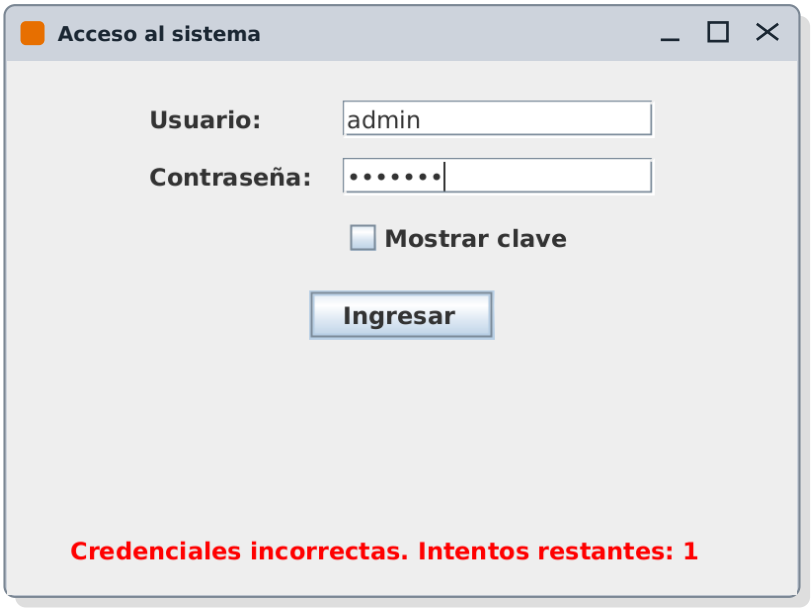


Figura 5. Formulario del ejercicio 5 en ejecución.

Solución · Ej05Login.java

```
import javax.swing.*;
import java.awt.*;
import java.util.Arrays;

public class Ej05Login extends JFrame {

    private static final String USUARIO = "admin";
    private static final String CLAVE = "1234";
    private static final int MAX_INTENTOS = 3;

    private final JTextField txtUsuario = new JTextField(14);
    private final JPasswordField txtClave = new JPasswordField(14);
    private final JCheckBox chkMostrar = new JCheckBox("Mostrar clave");
    private final JButton btnIngresar = new JButton("Ingresar");
    private final JLabel lblEstado =
            new JLabel("Tiene " + MAX_INTENTOS + " intentos", SwingConstants.CENTER);
    private final char echoOriginal = txtClave.getEchoChar();
    private int intentos = 0;

    public Ej05Login() {
        super("Acceso al sistema");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
```

```
        JPanel form = new JPanel(new GridBagLayout());
        form.setBorder(BorderFactory.createEmptyBorder(15, 20, 5, 20));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(5, 5, 5, 5);
        gbc.anchor = GridBagConstraints.WEST;

        gbc.gridx = 0; gbc.gridy = 0; form.add(new JLabel("Usuario:"), gbc);
        gbc.gridx = 1;                form.add(txtUsuario, gbc);
        gbc.gridx = 0; gbc.gridy = 1; form.add(new JLabel("Contraseña:"), gbc);
        gbc.gridx = 1;                form.add(txtClave, gbc);
        gbc.gridy = 2;                form.add(chkMostrar, gbc);

        JPanel panelBoton = new JPanel();
        panelBoton.add(btnIngresar);
        lblEstado.setBorder(BorderFactory.createEmptyBorder(0, 10, 15, 10));

        add(form, BorderLayout.NORTH);
        add(panelBoton, BorderLayout.CENTER);
        add(lblEstado, BorderLayout.SOUTH);

        btnIngresar.addActionListener(e -> ingresar());
        txtClave.addActionListener(e -> ingresar());          // Enter en la contraseña
        chkMostrar.addActionListener(e ->
                txtClave.setEchoChar(chkMostrar.isSelected() ? (char) 0 : echoOriginal));

        setSize(400, 270);
        setLocationRelativeTo(null);
    }

    private void ingresar() {
        String usuario = txtUsuario.getText().trim();
        char[] clave = txtClave.getPassword();
        boolean valido = usuario.equals(USUARIO) && new String(clave).equals(CLAVE);
        Arrays.fill(clave, '0');            // buena práctica: no dejar la clave en memoria

        if (valido) {
            JOptionPane.showMessageDialog(this, "¡Bienvenido, " + usuario + "!",
                    "Acceso concedido", JOptionPane.INFORMATION_MESSAGE);
            dispose();                      // aquí se abriría la ventana principal
            return;
        }

        intentos++;
        int restantes = MAX_INTENTOS - intentos;
        lblEstado.setForeground(Color.RED);
        if (restantes <= 0) {
            lblEstado.setText("Cuenta bloqueada por demasiados intentos");
            btnIngresar.setEnabled(false);
            txtUsuario.setEnabled(false);
            txtClave.setEnabled(false);
        } else {
            lblEstado.setText("Credenciales incorrectas. Intentos restantes: " + restantes);
            txtClave.setText("");
            txtClave.requestFocusInWindow();
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Ej05Login().setVisible(true));
    }
}
```

¿Cómo funciona?

- `getPassword()` devuelve un `char[]` y no un `String`: así se puede borrar de memoria con `Arrays.fill` apenas se usa.
- `setEchoChar((char) 0)` muestra el texto plano; se restaura con el carácter original guardado en `echoOriginal`.
- El contador `intentos` es un atributo de la ventana, por eso conserva su valor entre clics del botón.
- `setEnabled(false)` bloquea los componentes sin ocultarlos; el usuario ve claramente que el formulario dejó de estar disponible.

EJERCICIO 06

Pedido de pizzería con boleta

Temática: Gastronomía y ventas

Enunciado

Una pizzería necesita una pantalla para tomar pedidos: se elige el tamaño, se marcan ingredientes extra y se define la cantidad. Al presionar Calcular se imprime una boleta detallada con IVA incluido.

Requisitos

- Tamaño con `JRadioButton` (solo uno) y extras con `JCheckBox` (varios a la vez).
- Cantidad con `JSpinner` de 1 a 20.
- Boleta en un `JTextArea` de solo lectura con fuente monoespaciada y columnas alineadas.
- Mostrar precio unitario, cantidad, IVA incluido (19 %) y total. Botón **Nuevo pedido** para reiniciar.

Conceptos que se aplican: JRadioButton, ButtonGroup, JCheckBox, JSpinner, JTextArea, String.format, TitledBorder

Vista esperada del formulario

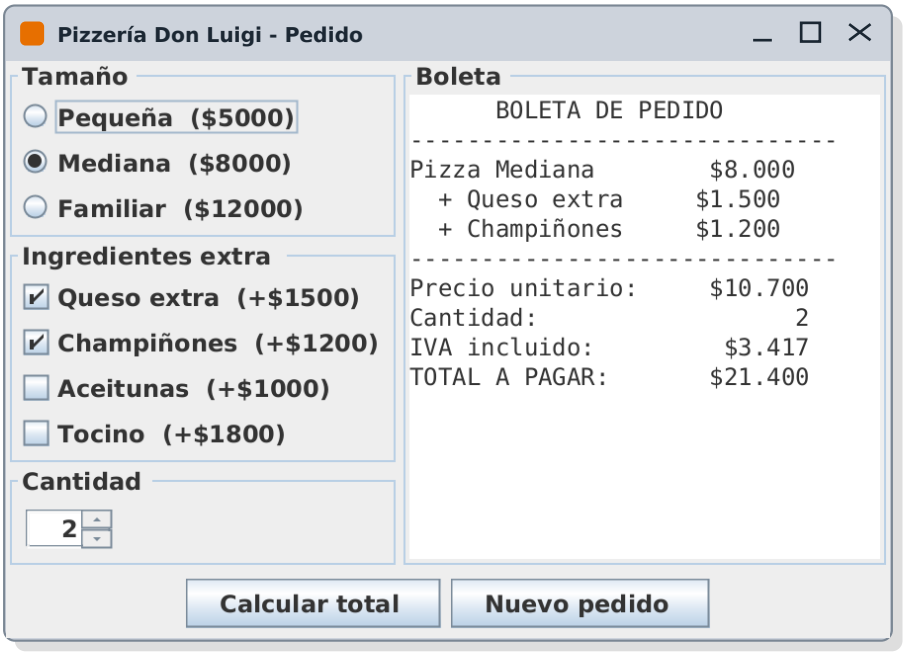


Figura 6. Formulario del ejercicio 6 en ejecución.

Solución · Ej06Pizzeria.java

```java
import javax.swing.*;
import java.awt.*;

public class Ej06Pizzeria extends JFrame {

    private static final String[] TAMANOS = {"Pequeña", "Mediana", "Familiar"};
    private static final int[] PRECIO_TAMANO = {5000, 8000, 12000};
    private static final String[] EXTRAS = {"Queso extra", "Champiñones", "Aceitunas", "Tocino"};
    private static final int[] PRECIO_EXTRA = {1500, 1200, 1000, 1800};

    private final JRadioButton[] radios = new JRadioButton[TAMANOS.length];
    private final JCheckBox[] checks = new JCheckBox[EXTRAS.length];
    private final JSpinner spnCantidad = new JSpinner(new SpinnerNumberModel(1, 1, 20, 1));
    private final JTextArea txtBoleta = new JTextArea(12, 34);

    public Ej06Pizzeria() {
        super("Pizzería Don Luigi - Pedido");
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        // --- Panel de tamaños (RadioButton: solo uno seleccionado) ---
        JPanel pnlTamano = new JPanel(new GridLayout(0, 1));
        pnlTamano.setBorder(BorderFactory.createTitledBorder("Tamaño"));
        ButtonGroup grupo = new ButtonGroup();
        for (int i = 0; i < TAMANOS.length; i++) {
            radios[i] = new JRadioButton(TAMANOS[i] + "  ($" + PRECIO_TAMANO[i] + ")");
            grupo.add(radios[i]);
            pnlTamano.add(radios[i]);
        }
        radios[0].setSelected(true);

        // --- Panel de extras (CheckBox: varios a la vez) ---
        JPanel pnlExtras = new JPanel(new GridLayout(0, 1));
        pnlExtras.setBorder(BorderFactory.createTitledBorder("Ingredientes extra"));
        for (int i = 0; i < EXTRAS.length; i++) {
            checks[i] = new JCheckBox(EXTRAS[i] + "  (+$" + PRECIO_EXTRA[i] + ")");
            pnlExtras.add(checks[i]);
        }

        JPanel pnlCantidad = new JPanel(new FlowLayout(FlowLayout.LEFT));
        pnlCantidad.setBorder(BorderFactory.createTitledBorder("Cantidad"));
        pnlCantidad.add(spnCantidad);

        JPanel izquierda = new JPanel(new BorderLayout());
        izquierda.add(pnlTamano, BorderLayout.NORTH);
        JPanel centroIzq = new JPanel(new BorderLayout());
        centroIzq.add(pnlExtras, BorderLayout.NORTH);
        centroIzq.add(pnlCantidad, BorderLayout.CENTER);
        izquierda.add(centroIzq, BorderLayout.CENTER);

        txtBoleta.setEditable(false);
        txtBoleta.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
        JScrollPane scroll = new JScrollPane(txtBoleta);
        scroll.setBorder(BorderFactory.createTitledBorder("Boleta"));

        JButton btnCalcular = new JButton("Calcular total");
        JButton btnNuevo = new JButton("Nuevo pedido");
        JPanel botones = new JPanel();
        botones.add(btnCalcular);
        botones.add(btnNuevo);

        add(izquierda, BorderLayout.WEST);
        add(scroll, BorderLayout.CENTER);
        add(botones, BorderLayout.SOUTH);

        btnCalcular.addActionListener(e -> calcular());
        btnNuevo.addActionListener(e -> nuevoPedido());

        pack();
        setLocationRelativeTo(null);
    }

    private String dinero(int monto) {
        return "$" + String.format("%,d", monto).replace(',', '.');
```

```
    }

    private void calcular() {
        int tamano = 0;
        for (int i = 0; i < radios.length; i++) {
            if (radios[i].isSelected()) tamano = i;
        }
        int unitario = PRECIO_TAMANO[tamano];

        StringBuilder sb = new StringBuilder();
        sb.append("      BOLETA DE PEDIDO\n");
        sb.append("------------------------------\n");
        sb.append(String.format("Pizza %-10s %10s%n", TAMANOS[tamano], dinero(unitario)));
        for (int i = 0; i < checks.length; i++) {
            if (checks[i].isSelected()) {
                unitario += PRECIO_EXTRA[i];
                sb.append(String.format("  + %-11s %10s%n", EXTRAS[i], dinero(PRECIO_EXTRA[i])));
            }
        }
        int cantidad = (int) spnCantidad.getValue();
        int total = unitario * cantidad;
        long iva = total - Math.round(total / 1.19);

        sb.append("------------------------------\n");
        sb.append(String.format("Precio unitario:  %10s%n", dinero(unitario)));
        sb.append(String.format("Cantidad:         %10d%n", cantidad));
        sb.append(String.format("IVA incluido:     %10s%n", dinero((int) iva)));
        sb.append(String.format("TOTAL A PAGAR:    %10s%n", dinero(total)));
        txtBoleta.setText(sb.toString());
    }

    private void nuevoPedido() {
        radios[0].setSelected(true);
        for (JCheckBox chk : checks) chk.setSelected(false);
        spnCantidad.setValue(1);
        txtBoleta.setText("");
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Ej06Pizzeria().setVisible(true));
    }
}
```

¿Cómo funciona?

- `ButtonGroup` hace que los radios sean **mutuamente excluyentes**; los `JCheckBox`, en cambio, son independientes.
- Los precios se guardan en **arreglos paralelos** (`TAMANOS` / `PRECIO_TAMANO`), evitando una cadena de `if` y facilitando agregar productos.
- `String.format("%-10s %10s")` alinea columnas; funciona porque el `JTextArea` usa una fuente monoespaciada.
- IVA incluido = total − total / 1,19. El método `dinero()` formatea con separador de miles.

EJERCICIO **07**

Monitor de hilos con JTextArea

Temática: Concurrencia y sistemas

HILOS + MONITOR (JTextArea)

Enunciado

La verdulería “Al Paso” tiene tres cámaras frigoríficas (frutas, verduras y lácteos). Cada cámara posee un sensor que entrega una lectura de temperatura a intervalos irregulares. Construye una ventana que ejecute **un hilo por sensor** y muestre todas las lecturas, en orden y sin mezclarse, en un **monitor** (`JTextArea` con estilo de consola). Debe alertar cuando una cámara supere los 8 °C.

Requisitos

- Tres hilos (`Thread`), uno por cámara, con una espera aleatoria entre 0,5 y 1,5 s entre lecturas.
- Un método `registrar()` **sincronizado** que escriba en el monitor y cuente los eventos (recurso compartido).
- Actualizar el `JTextArea` **solo desde el hilo de eventos de Swing** (EDT), con auto-scroll.
- Botones **Iniciar monitoreo**, **Detener** (interrumpe los hilos) y **Limpiar**; barra de estado con el estado y el total de eventos.
- Alerta visible cuando la temperatura sea mayor que 8 °C.

Conceptos que se aplican: Thread, Runnable (lambda), synchronized, Object como cerrojo, SwingUtilities.invokeLater, interrupt(), JTextArea, JScrollPane

Vista esperada del formulario



Figura 7. Formulario del ejercicio 7 en ejecución.

Solución · Ej07MonitorHilos.java

```
import javax.swing.*;
import java.awt.*;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class Ej07MonitorHilos extends JFrame {

    private static final DateTimeFormatter HORA = DateTimeFormatter.ofPattern("HH:mm:ss");
    private static final String[] CAMARAS = {"Cámara Frutas", "Cámara Verduras", "Cámara Lácteos"};
```

```
    private final JTextArea txtMonitor = new JTextArea(14, 72);
    private final JLabel lblEstado = new JLabel("Estado: DETENIDO");
    private final JLabel lblEventos = new JLabel("Eventos: 0");
    private final JButton btnIniciar = new JButton("Iniciar monitoreo");
    private final JButton btnDetener = new JButton("Detener");
    private final JButton btnLimpiar = new JButton("Limpiar");

    private final List<Thread> sensores = new ArrayList<>();
    private final Object cerrojo = new Object();   // objeto "monitor" que protege la sección crítica
    private int totalEventos = 0;                  // recurso compartido entre los hilos

    public Ej07MonitorHilos() {
        super("Monitor de hilos - Cámaras de frío");
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        txtMonitor.setEditable(false);
        txtMonitor.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
        txtMonitor.setBackground(Color.BLACK);
        txtMonitor.setForeground(new Color(0, 230, 0));

        JPanel barra = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 6));
        barra.add(btnIniciar);
        barra.add(btnDetener);
        barra.add(btnLimpiar);

        JPanel estado = new JPanel(new BorderLayout());
        estado.setBorder(BorderFactory.createEmptyBorder(4, 10, 4, 10));
        estado.add(lblEstado, BorderLayout.WEST);
        estado.add(lblEventos, BorderLayout.EAST);

        add(barra, BorderLayout.NORTH);
        JScrollPane scroll = new JScrollPane(txtMonitor,
                JScrollPane.VERTICAL_SCROLLBAR_ALWAYS, JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);
        add(scroll, BorderLayout.CENTER);
        add(estado, BorderLayout.SOUTH);

        btnDetener.setEnabled(false);
        btnIniciar.addActionListener(e -> iniciar());
        btnDetener.addActionListener(e -> detener());
        btnLimpiar.addActionListener(e -> txtMonitor.setText(""));

        pack();
        setLocationRelativeTo(null);
    }

    /** Sección crítica: solo un hilo a la vez puede ejecutar el bloque synchronized. */
    private void registrar(String mensaje, boolean esLectura) {
        synchronized (cerrojo) {
            if (esLectura) {
                totalEventos++;
            }
            String linea = LocalTime.now().format(HORA) + "  " + mensaje + "\n";
            int total = totalEventos;
            // Swing NO es thread-safe: la actualización visual se delega al hilo de eventos (EDT)
            SwingUtilities.invokeLater(() -> {
                txtMonitor.append(linea);
                txtMonitor.setCaretPosition(txtMonitor.getDocument().getLength()); // auto-scroll
                lblEventos.setText("Eventos: " + total);
            });
        }
    }

    private void iniciar() {
        registrar("Monitoreo INICIADO con " + CAMARAS.length + " hilos", false);
        for (String camara : CAMARAS) {
            Thread hilo = new Thread(() -> {
                Random azar = new Random();
                while (!Thread.currentThread().isInterrupted()) {
                    double temp = 2 + azar.nextDouble() * 8;              // entre 2,0 y 10,0 °C
                    String alerta = temp > 8 ? "  <-- ALERTA: temperatura alta" : "";
                    registrar(String.format("%-16s %4.1f °C%s", camara, temp, alerta), true);
                    try {
                        Thread.sleep(500 + azar.nextInt(1000));           // espera de 0,5 a 1,5 s
                    } catch (InterruptedException ex) {
                        Thread.currentThread().interrupt();               // termina el while
```

```
                    }
                }
            }, "Sensor-" + camara);
            hilo.setDaemon(true);
            sensores.add(hilo);
            hilo.start();
        }
        lblEstado.setText("Estado: ACTIVO (" + CAMARAS.length + " hilos)");
        btnIniciar.setEnabled(false);
        btnDetener.setEnabled(true);
    }

    private void detener() {
        for (Thread hilo : sensores) {
            hilo.interrupt();                    // pide a cada hilo que termine
        }
        sensores.clear();
        registrar("Monitoreo DETENIDO", false);
        lblEstado.setText("Estado: DETENIDO");
        btnIniciar.setEnabled(true);
        btnDetener.setEnabled(false);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Ej07MonitorHilos().setVisible(true));
    }
}
```

Salida esperada en el monitor

Fragmento real de una ejecución. Las tres cámaras escriben con ritmos distintos, y las líneas siempre salen completas y en orden gracias al bloque `synchronized`. Las lecturas por encima de 8 °C generan una alerta.

```
19:38:40  Monitoreo INICIADO con 3 hilos
19:38:40  Cámara Frutas     6.1 °C
19:38:40  Cámara Verduras   6.4 °C
19:38:40  Cámara Lácteos    8.1 °C  <-- ALERTA: temperatura alta
19:38:41  Cámara Verduras   3.9 °C
19:38:41  Cámara Lácteos    2.0 °C
19:38:42  Cámara Frutas     6.5 °C
19:38:42  Cámara Lácteos    9.4 °C  <-- ALERTA: temperatura alta
19:38:42  Cámara Verduras   4.1 °C
19:38:43  Cámara Lácteos    8.2 °C  <-- ALERTA: temperatura alta
19:38:43  Cámara Frutas     5.1 °C
19:38:43  Cámara Lácteos    5.6 °C
19:38:44  Cámara Verduras   8.4 °C  <-- ALERTA: temperatura alta
19:38:44  Cámara Verduras   7.2 °C
```

Flujo de datos entre los hilos y el monitor



Los hilos NUNCA tocan el JTextArea directamente: solo lo hace el EDT, gracias a invokeLater.

¿Cómo funciona?

- **Un hilo por sensor:** cada `Thread` corre su propio `while` y duerme con `Thread.sleep`, por eso las lecturas llegan desordenadas en el tiempo, como en un sistema real.
- **El monitor (sincronización):** el bloque `synchronized (cerrojo)` de `registrar()` es la sección crítica. Solo un hilo la ejecuta a la vez, así `totalEventos++` no sufre condiciones de carrera y cada línea se arma completa.
- **Swing no es thread-safe:** los hilos de trabajo no llaman a `txtMonitor.append()`. Encolan la tarea con `SwingUtilities.invokeLater`, y el EDT la ejecuta en orden (cola FIFO).
- **Detención cooperativa:** `interrupt()` despierta el `sleep` con `InterruptedException`; el `catch` restaura la marca de interrupción y el `while` termina. Nunca se usa `Thread.stop()`.
- `setDaemon(true)` evita que los hilos impidan cerrar la aplicación. `setCaretPosition` al final del documento logra el auto-scroll.
- **Regla de oro:** jamás poner `Thread.sleep` ni tareas largas dentro de un `ActionListener`; congelarían la ventana porque corren en el EDT.

Para practicar (extensiones)

- Agrega un cuarto sensor (Cámara Congelados) y un `JComboBox` para elegir la frecuencia de lectura.
- Cuenta las alertas en otra variable protegida por el mismo `cerrojo` y muéstrala en la barra de estado.
- Reemplaza `hilo.interrupt()` por un `volatile boolean activo` y compara ambas formas de detener un hilo.
- Implementa un patrón **productor/consumidor**: los sensores encolan lecturas y un único hilo consumidor las escribe en el monitor.

EJERCICIO
08

Juego: adivina el número

Temática: Videojuegos y lógica

Enunciado

El programa elige un número secreto entre 1 y 100. El jugador tiene 7 intentos para adivinarlo y recibe pistas de “mayor” o “menor” en cada intento. Una barra de progreso muestra los intentos utilizados.

Requisitos

- `JSpinner` de 1 a 100 y botón **Probar**.
- Pista dinámica en el título (mayor o menor que el número probado).
- `JProgressBar` con texto “Intentos: n / 7”.
- Victoria o derrota bloquea los controles; botón **Nuevo juego** reinicia todo.

Conceptos que se aplican: Random, JSpinner, JProgressBar, estado del juego, UIManager.getColor(), setEnabled()

Vista esperada del formulario

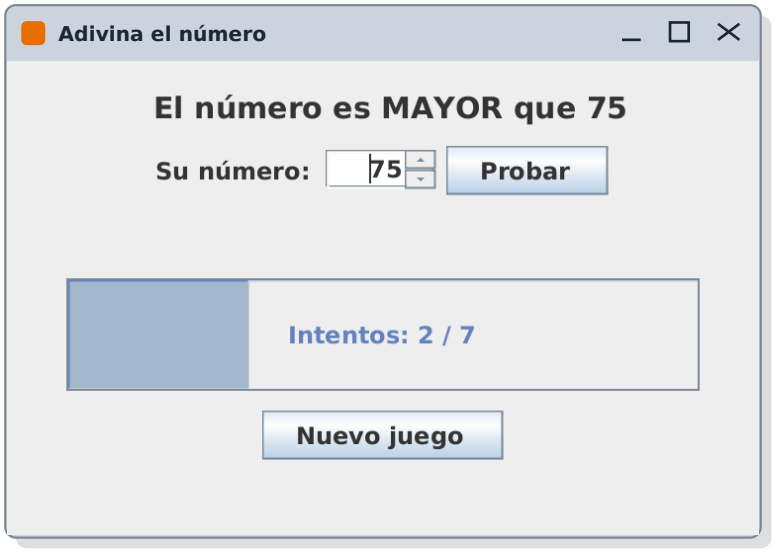


Figura 8. Formulario del ejercicio 8 en ejecución.

Solución · Ej08Adivina.java

```
import javax.swing.*;
import java.awt.*;
import java.util.Random;

public class Ej08Adivina extends JFrame {

    private static final int MAX_INTENTOS = 7;

    private final Random azar = new Random();
    private int secreto;
    private int intentos;

    private final JLabel lblPista = new JLabel("", SwingConstants.CENTER);
    private final JSpinner spnNumero = new JSpinner(new SpinnerNumberModel(50, 1, 100, 1));
    private final JButton btnProbar = new JButton("Probar");
    private final JButton btnNuevo = new JButton("Nuevo juego");
    private final JProgressBar barra = new JProgressBar(0, MAX_INTENTOS);

    public Ej08Adivina() {
        super("Adivina el número");
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        lblPista.setFont(lblPista.getFont().deriveFont(Font.BOLD, 15f));
        lblPista.setBorder(BorderFactory.createEmptyBorder(15, 10, 5, 10));
```

```
        JPanel fila1 = new JPanel();
        fila1.add(new JLabel("Su número:"));
        fila1.add(spnNumero);
        fila1.add(btnProbar);

        barra.setStringPainted(true);
        JPanel fila2 = new JPanel(new BorderLayout());
        fila2.setBorder(BorderFactory.createEmptyBorder(5, 30, 5, 30));
        fila2.add(barra, BorderLayout.CENTER);

        JPanel fila3 = new JPanel();
        fila3.add(btnNuevo);

        JPanel centro = new JPanel(new GridLayout(3, 1));
        centro.add(fila1);
        centro.add(fila2);
        centro.add(fila3);

        add(lblPista, BorderLayout.NORTH);
        add(centro, BorderLayout.CENTER);

        btnProbar.addActionListener(e -> probar());
        btnNuevo.addActionListener(e -> nuevoJuego());

        nuevoJuego();
        setSize(380, 240);
        setLocationRelativeTo(null);
    }

    private void nuevoJuego() {
        secreto = azar.nextInt(100) + 1;          // número entre 1 y 100
        intentos = 0;
        barra.setValue(0);
        barra.setString("Intentos: 0 / " + MAX_INTENTOS);
        lblPista.setForeground(UIManager.getColor("Label.foreground"));
        lblPista.setText("Adivine el número entre 1 y 100");
        spnNumero.setEnabled(true);
        btnProbar.setEnabled(true);
    }

    private void probar() {
        int n = (int) spnNumero.getValue();
        intentos++;
        barra.setValue(intentos);
        barra.setString("Intentos: " + intentos + " / " + MAX_INTENTOS);

        if (n == secreto) {
            terminar("¡Correcto! Lo logró en " + intentos + " intentos", new Color(0, 140, 0));
        } else if (intentos >= MAX_INTENTOS) {
            terminar("Perdió. El número era " + secreto, Color.RED);
        } else {
            lblPista.setText(n < secreto ? "El número es MAYOR que " + n
                                         : "El número es MENOR que " + n);
        }
    }

    private void terminar(String mensaje, Color color) {
        lblPista.setForeground(color);
        lblPista.setText(mensaje);
        spnNumero.setEnabled(false);
        btnProbar.setEnabled(false);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Ej08Adivina().setVisible(true));
    }
}
```

¿Cómo funciona?

- El estado del juego (`secreto`, `intentos`) son atributos: sobreviven entre un clic y otro, algo que una variable local no lograría.
- `azar.nextInt(100) + 1` produce un entero entre 1 y 100 (el rango de `nextInt(100)` es 0 a 99).
- `nuevoJuego()` se invoca desde el constructor y desde el botón: **una sola función** evita duplicar la inicialización.
- `UIManager.getColor("Label.foreground")` recupera el color original de la etiqueta tras una victoria o derrota.

EJERCICIO
09

Mini editor de texto

Temática: Ofimática y archivos

Enunciado

Desarrolla un editor de texto básico con menús: crear un documento nuevo, abrir y guardar archivos `.txt`, activar el ajuste de línea y cambiar el tamaño de la fuente. Una barra de estado debe mostrar líneas y caracteres en tiempo real.

Requisitos

- `JMenuBar` con los menús **Archivo** (Nuevo, Abrir, Guardar, Salir) y **Formato**.
- Atajos de teclado Ctrl+N, Ctrl+O, Ctrl+S y Ctrl+Q.
- `JFileChooser` para abrir y guardar; manejo de errores de lectura y escritura.
- `DocumentListener` para actualizar la barra de estado a cada tecla.

Conceptos que se aplican: JMenuBar, JMenu, JMenuItem, JCheckBoxMenuItem, KeyStroke, JFileChooser, Files, DocumentListener

Vista esperada del formulario

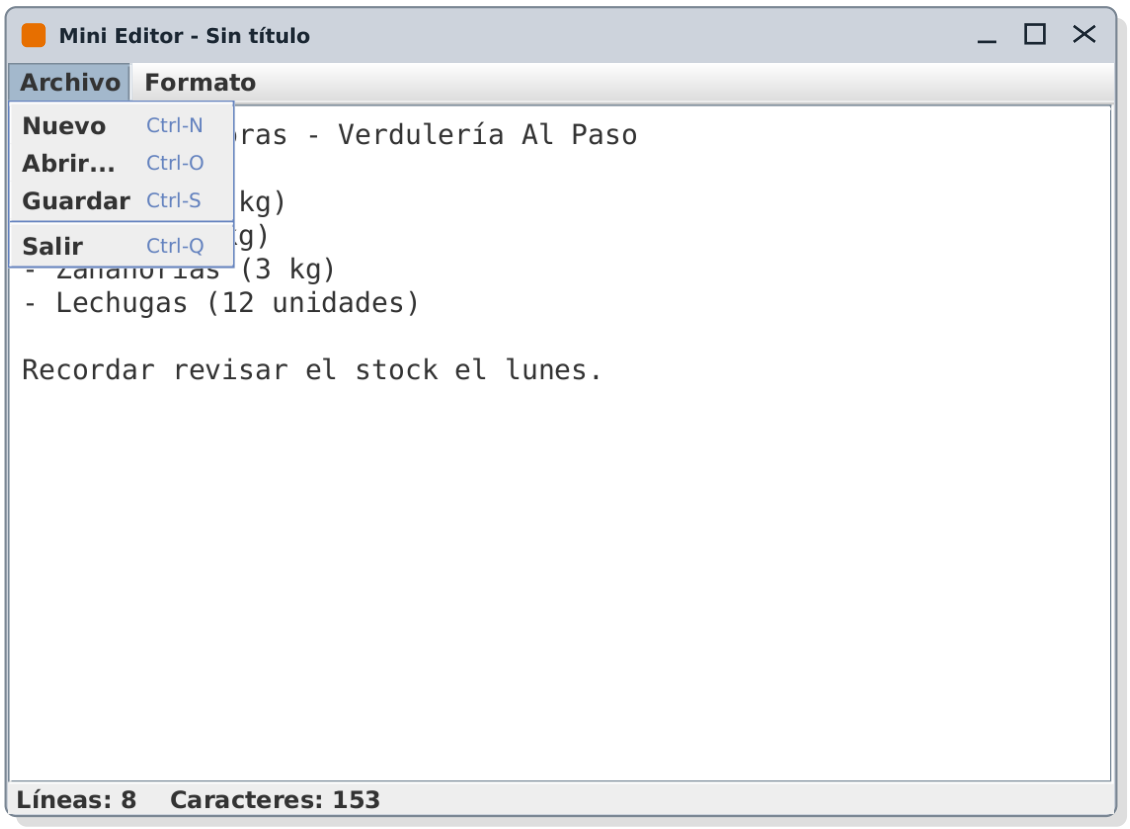


Figura 9. Formulario del ejercicio 9 en ejecución.

Solución · Ej09Editor.java

```
import javax.swing.*;
import javax.swing.event.DocumentEvent;
import javax.swing.event.DocumentListener;
import java.awt.*;
import java.awt.event.ActionListener;
import java.awt.event.KeyEvent;
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;

public class Ej09Editor extends JFrame {

    private final JTextArea areaTexto = new JTextArea();
    private final JLabel lblEstado = new JLabel();
    private File archivoActual = null;

    public Ej09Editor() {
        super("Mini Editor - Sin título");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(560, 380);
        setLocationRelativeTo(null);

        areaTexto.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 14));
        areaTexto.setMargin(new Insets(6, 6, 6, 6));
        add(new JScrollPane(areaTexto), BorderLayout.CENTER);
        add(lblEstado, BorderLayout.SOUTH);
        setJMenuBar(crearMenu());

        // Cada cambio en el documento actualiza la barra de estado
        areaTexto.getDocument().addDocumentListener(new DocumentListener() {
            public void insertUpdate(DocumentEvent e)  { actualizarEstado(); }
            public void removeUpdate(DocumentEvent e)  { actualizarEstado(); }
            public void changedUpdate(DocumentEvent e) { actualizarEstado(); }
        });
        actualizarEstado();
    }

    private JMenuBar crearMenu() {
        JMenu mnuArchivo = new JMenu("Archivo");
        mnuArchivo.add(crearItem("Nuevo", KeyEvent.VK_N, e -> nuevo()));
        mnuArchivo.add(crearItem("Abrir...", KeyEvent.VK_O, e -> abrir()));
        mnuArchivo.add(crearItem("Guardar", KeyEvent.VK_S, e -> guardar()));
        mnuArchivo.addSeparator();
        mnuArchivo.add(crearItem("Salir", KeyEvent.VK_Q, e -> System.exit(0)));

        JMenu mnuFormato = new JMenu("Formato");
        JCheckBoxMenuItem itmAjuste = new JCheckBoxMenuItem("Ajuste de línea");
        itmAjuste.addActionListener(e -> {
            areaTexto.setLineWrap(itmAjuste.isSelected());
            areaTexto.setWrapStyleWord(true);
        });
        JMenuItem itmMas = new JMenuItem("Aumentar fuente");
        JMenuItem itmMenos = new JMenuItem("Reducir fuente");
        itmMas.addActionListener(e -> cambiarFuente(+2));
        itmMenos.addActionListener(e -> cambiarFuente(-2));
        mnuFormato.add(itmAjuste);
        mnuFormato.addSeparator();
        mnuFormato.add(itmMas);
        mnuFormato.add(itmMenos);

        JMenuBar barra = new JMenuBar();
        barra.add(mnuArchivo);
        barra.add(mnuFormato);
        return barra;
    }

    private JMenuItem crearItem(String texto, int tecla, ActionListener accion) {
        JMenuItem item = new JMenuItem(texto);
        item.setAccelerator(KeyStroke.getKeyStroke(tecla,
                Toolkit.getDefaultToolkit().getMenuShortcutKeyMaskEx()));   // Ctrl + tecla
        item.addActionListener(accion);
        return item;
    }
```

```
    private void nuevo() {
        areaTexto.setText("");
        archivoActual = null;
        setTitle("Mini Editor - Sin título");
    }

    private void abrir() {
        JFileChooser selector = new JFileChooser();
        if (selector.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
            try {
                archivoActual = selector.getSelectedFile();
                areaTexto.setText(Files.readString(archivoActual.toPath()));
                setTitle("Mini Editor - " + archivoActual.getName());
            } catch (IOException ex) {
                mostrarError("No se pudo abrir el archivo: " + ex.getMessage());
            }
        }
    }

    private void guardar() {
        if (archivoActual == null) {
            JFileChooser selector = new JFileChooser();
            if (selector.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) return;
            archivoActual = selector.getSelectedFile();
        }
        try {
            Files.writeString(archivoActual.toPath(), areaTexto.getText());
            setTitle("Mini Editor - " + archivoActual.getName());
        } catch (IOException ex) {
            mostrarError("No se pudo guardar el archivo: " + ex.getMessage());
        }
    }

    private void cambiarFuente(int delta) {
        Font f = areaTexto.getFont();
        areaTexto.setFont(f.deriveFont(f.getSize2D() + delta));
    }

    private void actualizarEstado() {
        lblEstado.setText(" Líneas: " + areaTexto.getLineCount()
                + "    Caracteres: " + areaTexto.getDocument().getLength());
    }

    private void mostrarError(String mensaje) {
        JOptionPane.showMessageDialog(this, mensaje, "Error", JOptionPane.ERROR_MESSAGE);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Ej09Editor().setVisible(true));
    }
}
```

¿Cómo funciona?

- El método `crearItem()` **evita repetir** código: crea el ítem, asigna el atajo y registra la acción en una sola llamada.
- `getMenuShortcutKeyMaskEx()` usa Ctrl en Windows/Linux y Cmd en macOS automáticamente.
- `Files.readString` y `Files.writeString` (Java 11+) leen y escriben archivos completos en una línea; ambos lanzan `IOException`, que se muestra con `JOptionPane`.
- `DocumentListener` reacciona a cada inserción o eliminación del texto: es la forma correcta de detectar cambios, en lugar de sondear con un temporizador.

EJERCICIO
10

Semáforo animado

Temática: Gráficos y animación

Enunciado

Dibuja un semáforo con tres luces que cambian automáticamente: rojo 4 s, verde 4 s y amarillo 2 s, en ciclo. Debe poder pausarse, reanudarse y reiniciarse, y mostrar el estado actual con los segundos restantes.

Requisitos

- Panel personalizado que dibuje el semáforo con `paintComponent` y `Graphics2D`.
- `enum Estado` con texto, duración y método `siguiente()`.
- `javax.swing.Timer` de 1 segundo para la animación.
- Botones **Iniciar / Pausar / Reanudar** y **Reiniciar**.

Conceptos que se aplican: paintComponent, Graphics2D, antialiasing, enum con atributos, switch expression, javax.swing.Timer, repaint()

Vista esperada del formulario

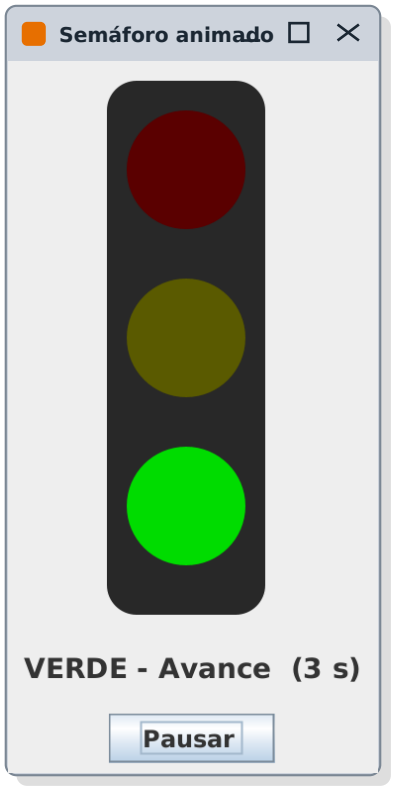


Figura 10. Formulario del ejercicio 10 en ejecución.

Solución · Ej10Semaforo.java

```
import javax.swing.*;
import java.awt.*;

public class Ej10Semaforo extends JFrame {

    /** Cada estado conoce su texto, su duración y cuál es el siguiente. */
    private enum Estado {
        ROJO("ROJO - Deténgase", 4),
        VERDE("VERDE - Avance", 4),
        AMARILLO("AMARILLO - Precaución", 2);

        final String texto;
```

```
        final int segundos;

        Estado(String texto, int segundos) {
            this.texto = texto;
            this.segundos = segundos;
        }

        Estado siguiente() {
            return switch (this) {
                case ROJO -> VERDE;
                case VERDE -> AMARILLO;
                case AMARILLO -> ROJO;
            };
        }
    }

    /** Panel personalizado: dibuja el semáforo sobreescribiendo paintComponent. */
    private class PanelSemaforo extends JPanel {
        PanelSemaforo() {
            setPreferredSize(new Dimension(180, 290));
        }

        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
            g2.setColor(new Color(40, 40, 40));
            g2.fillRoundRect(50, 10, 80, 270, 30, 30);
            dibujarLuz(g2, 25, Estado.ROJO, Color.RED, new Color(90, 0, 0));
            dibujarLuz(g2, 110, Estado.AMARILLO, Color.YELLOW, new Color(90, 90, 0));
            dibujarLuz(g2, 195, Estado.VERDE, new Color(0, 220, 0), new Color(0, 70, 0));
            g2.dispose();
        }

        private void dibujarLuz(Graphics2D g2, int y, Estado luz, Color on, Color off) {
            g2.setColor(estado == luz ? on : off);     // enciende solo la luz del estado actual
            g2.fillOval(60, y, 60, 60);
        }
    }

    private Estado estado = Estado.ROJO;
    private int restante = estado.segundos;

    private final PanelSemaforo panel = new PanelSemaforo();
    private final JLabel lblEstado = new JLabel("", SwingConstants.CENTER);
    private final JButton btnIniciar = new JButton("Iniciar");
    private final JButton btnReiniciar = new JButton("Reiniciar");
    private final Timer timer = new Timer(1000, e -> tick());   // dispara cada 1 segundo

    public Ej10Semaforo() {
        super("Semáforo animado");
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        lblEstado.setFont(lblEstado.getFont().deriveFont(Font.BOLD, 14f));
        JPanel botones = new JPanel();
        botones.add(btnIniciar);
        botones.add(btnReiniciar);
        JPanel sur = new JPanel(new GridLayout(2, 1));
        sur.add(lblEstado);
        sur.add(botones);

        add(panel, BorderLayout.CENTER);
        add(sur, BorderLayout.SOUTH);

        btnIniciar.addActionListener(e -> alternar());
        btnReiniciar.addActionListener(e -> reiniciar());

        actualizar();
        pack();
        setLocationRelativeTo(null);
    }

    private void alternar() {
        if (timer.isRunning()) {
```

```
            timer.stop();
            btnIniciar.setText("Reanudar");
        } else {
            timer.start();
            btnIniciar.setText("Pausar");
        }
    }

    private void reiniciar() {
        timer.stop();
        estado = Estado.ROJO;
        restante = estado.segundos;
        btnIniciar.setText("Iniciar");
        actualizar();
    }

    private void tick() {
        restante--;
        if (restante <= 0) {
            estado = estado.siguiente();
            restante = estado.segundos;
        }
        actualizar();
    }

    private void actualizar() {
        lblEstado.setText(estado.texto + "  (" + restante + " s)");
        panel.repaint();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Ej10Semaforo().setVisible(true));
    }
}
```

¿Cómo funciona?

- Se sobrescribe `paintComponent` y **siempre** se llama primero a `super.paintComponent(g)` para limpiar el fondo; `repaint()` solicita un nuevo dibujado.
- Solo la luz del estado actual se dibuja encendida; las demás usan un tono oscuro, lo que simula el apagado.
- El `enum` concentra datos y comportamiento: cada estado sabe su duración y cuál es el siguiente (`switch` exhaustivo, sin `default`).
- `javax.swing.Timer` ejecuta su acción **en el EDT**, por lo que puede modificar componentes con seguridad. Es la alternativa más simple a un hilo cuando solo se necesita repetir una acción cada cierto tiempo.